

DBMS – REVIEW AND RATING MANAGEMENT SYSTEM

Project Title: Big Basket E-commerce Database Management System

1. Database Design

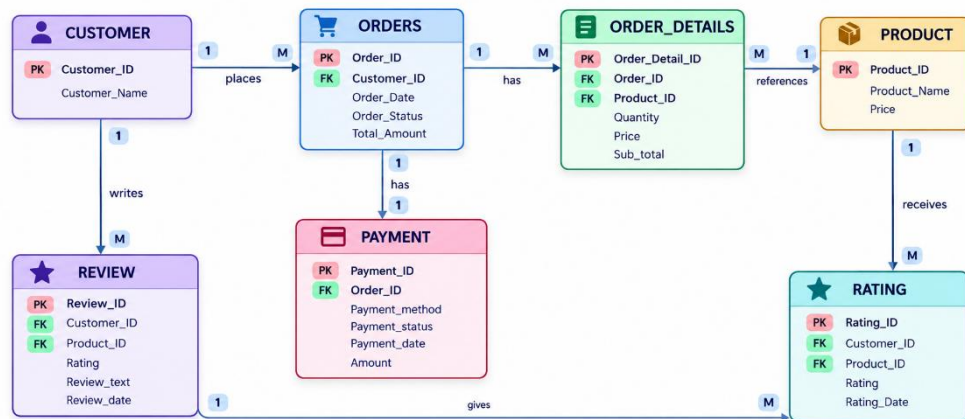
1.1 Rating Table

Attribute	Key Type
Rating_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	Not Null
Rating_Date	-

1.2 Review Table

Attribute	Key Type
Review_ID	Primary Key
Customer_ID	Foreign Key
Product_ID	Foreign Key
Rating	-
Review_text	-
Review_date	-

2. Primary and Foreign Key Relationship



3. Table Creation

3.1 Create Rating Table

```
CREATE TABLE Rating (  
    Rating_ID NUMBER PRIMARY KEY,  
    Customer_ID NUMBER NOT NULL,  
    Product_ID NUMBER NOT NULL,  
    Rating NUMBER(1) NOT NULL,  
    Rating_Date DATE,  
    CONSTRAINT fk_rating_customer  
        FOREIGN KEY (Customer_ID)  
        REFERENCES Customer(Customer_ID),  
    CONSTRAINT fk_rating_product  
        FOREIGN KEY (Product_ID)  
        REFERENCES Product(Product_ID),  
    CONSTRAINT chk_rating  
        CHECK (Rating BETWEEN 1 AND 5)  
);
```

The Rating table is created with Rating_ID as the primary key. It stores Rating details such as Customer ID, Product ID, Rating, and Rating Date.

3.2 Create Review Table

```
CREATE TABLE Review (  
    Review_ID NUMBER PRIMARY KEY,  
    Customer_ID NUMBER NOT NULL,  
    Product_ID NUMBER NOT NULL,  
    Rating NUMBER(1),  
    Review_text VARCHAR2(500),  
    Review_date DATE,  
    CONSTRAINT fk_review_customer  
        FOREIGN KEY (Customer_ID)  
        REFERENCES Customer(Customer_ID),  
    CONSTRAINT fk_review_product  
        FOREIGN KEY (Product_ID)  
        REFERENCES Product(Product_ID)  
);
```

The Review table is created with Review_ID as the primary key. Customer_ID and Product_ID are foreign keys that connect the review with the respective customer and product.

4. Data Insertion

4.1 Rating Records

```
INSERT INTO Rating
VALUES (1, 101, 101, 5, TO_DATE('20-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (2, 102, 102, 4, TO_DATE('21-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (3, 103, 103, 5, TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (4, 104, 104, 3, TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (5, 105, 105, 4, TO_DATE('23-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (6, 106, 106, 5, TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (7, 107, 107, 2, TO_DATE('24-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (8, 108, 108, 4, TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (9, 109, 109, 5, TO_DATE('25-09-2026','DD-MM-YYYY'));
1 row created.
```

```
INSERT INTO Rating
VALUES (10, 110, 110, 3, TO_DATE('26-09-2026','DD-MM-YYYY'));
1 row created.
```

4.2 Review Records

```

INSERT INTO Review
VALUES (1, 101, 101, 5, 'Excellent product and good quality', TO_DATE('20-09-2026','DD-MM-YYYY'));
1 row created.

INSERT INTO Review
VALUES (2, 102, 102, 4, 'Good product and worth the price', TO_DATE('21-09-2026','DD-MM-YYYY'));
1 row created.

INSERT INTO Review
VALUES (3, 103, 103, 5, 'Very useful and good quality', TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.

INSERT INTO Review
VALUES (4, 104, 104, 3, 'Average product', TO_DATE('22-09-2026','DD-MM-YYYY'));
1 row created.

INSERT INTO Review
VALUES (5, 105, 105, 4, 'Good product and fast delivery', TO_DATE('23-09-2026','DD-MM-YYYY'));
1 row created.

```

5. Data Retrieval

5.1 Display Rating Records

```

Select * from Rating;

```

RATING_ID	CUSTOMER_ID	PRODUCT_ID	RATING	RATING_DA
1	101	101	5	20-SEP-26
2	102	102	4	21-SEP-26
3	103	103	5	22-SEP-26
4	104	104	3	22-SEP-26
5	105	105	4	23-SEP-26
6	106	106	5	24-SEP-26
7	107	107	2	24-SEP-26
8	108	108	4	25-SEP-26
9	109	109	5	25-SEP-26
10	110	110	3	26-SEP-26

```

10 rows selected.

```

5.2 Display Review Records

```

Select * from Review;
REVIEW_ID CUSTOMER_ID PRODUCT_ID      RATING
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
|  |  | 1 |      101 |      101 |      5 |
Excellent product and good quality
20-SEP-26
|  |  | 2 |      102 |      102 |      4 |
Good product and worth the price
21-SEP-26
|  |  | 3 |      103 |      103 |      5 |
Very useful and good quality
22-SEP-26
|  |  | 4 |      104 |      104 |      3 |
Average product
22-SEP-26
|  |  | 5 |      105 |      105 |      4 |
Good product and fast delivery
23-SEP-26
5 rows selected.

```

6. Product Review Information

A JOIN query is used to display the customer name, product name, rating, review text, and review date. This query combines the Review, Customer, and Product tables to display complete product review information.

REVIEW_ID	CUSTOMER_NAME		
PRODUCT_NAME			RATING
REVIEW_TEXT			
REVIEW_DA			
	2 Priya Sharma		
Smartphone			4
Good product and worth the price			
21-SEP-26			
REVIEW_ID	CUSTOMER_NAME		
PRODUCT_NAME			RATING
REVIEW_TEXT			
REVIEW_DA			
	1 Arun Kumar		
Laptop			5
Excellent product and good quality			
20-SEP-26			

7. Calculate Average Product Ratings

This uses the AVG() aggregate function

```

Select Product_ID,Avg(Rating) as Average_Rating
From Rating
Group by Product_ID;

```

PRODUCT_ID	AVERAGE_RATING
101	5
102	4
103	5
104	3
105	4
106	5
107	2
108	4
109	5
110	3

10 rows selected.

8. Identify the Highly Rated Products

This report displays products that have received an average rating of 4 or above.

```

SELECT
    p.Product_ID,
    p.Product_Name,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Product p
JOIN Review r
    ON p.Product_ID = r.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
HAVING AVG(r.Rating) >= 4
ORDER BY
    Average_Rating DESC;

```

PRODUCT_ID	PRODUCT_NAME	AVERAGE_RATING
103	Tablet	5
101	Laptop	5
105	Mouse	4
102	Smartphone	4

9. Product Rating Report

```

SELECT
    p.Product_ID,
    p.Product_Name,
    COUNT(r.Review_ID) AS Total_Reviews,
    ROUND(AVG(r.Rating), 2) AS Average_Rating,
    MAX(r.Rating) AS Highest_Rating,
    MIN(r.Rating) AS Lowest_Rating
FROM Product p
JOIN Review r
    ON p.Product_ID = r.Product_ID
GROUP BY
    p.Product_ID,
    p.Product_Name
ORDER BY
    Average_Rating DESC;

```

PRODUCT_ID	PRODUCT_NAME	TOTAL_REVIEWS
101	Laptop	1
103	Tablet	1
105	Mouse	1

PRODUCT_ID	PRODUCT_NAME	TOTAL_REVIEWS		
AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING		
102	Smartphone	4	4	1
104	Keyboard	3	3	1

```

commit;
Commit complete

```

10. Conclusion

The **Review and Rating tables** were successfully created and connected with the existing **Customer and Product tables** using primary and foreign keys. Customer ratings and product reviews can be stored and retrieved efficiently. SQL queries can be used to display product review details, calculate average ratings, identify highly rated and lowly rated products, and generate complete product rating reports. These reports help analyse customer feedback and understand product performance based on customer ratings.